TECHNICAL SPECIFICATIONS
Hermes Platform
Courier Services Management and Tracking Platform



AUTHOR: Document Team
DATE:29/08/2026
Version 0.1.3

REVISION HISTORY			
DATE	**VERSION**	**DESCRIPTION**	**AUTHOR**
29/08/2026	0.1.0	Initial draft of the document	Cian Goeminne
30/08/2026	0.1.1	Completed Description, System Features, and External Interface sections (front-end input).	Aadil Ismail
31/08/2026	0.1.2	Review of all sections (Back-end input)	Jandre Burger
31/08/2026	0.1.3	Final edits and revisions for formatting	Cian Goeminne

Table of Contents

1. INTRODUCTION 3
1.1 PURPOSE 3
1.2 INTENDED AUDIENCE AND PERTINENT SECTIONS 3
1.3 PROJECT SCOPE 3
1.4 DOCUMENT CONVENTIONS 3
1.5 REFERENCES 4
2. DESCRIPTION 4
2.1 PRODUCT PERSPECTIVE 4
2.2 FEATURES 4
2.3 USER OVERVIEW 4
2.4 OPERATING ENVIRONMENT 5
2.5 CONTRAINTS: IMPLEMENTATION / DESIGN 6
2.6 DOCUMENTATION 6
2.7 ASSUMPTIONS / DEPENDENCIES 6
SYSTEM FEATURES 8
Parcel Management 8
Courier Management 8
Automated Notification System 9
3. REQUIREMENTS OF EXTERNAL INTERFACE 10
3.1 USER INTERFACES 10
3.2 HARDWARE INTERFACES 10
3.3 SOFTWARE INTERFACES 10
3.4 COMMUNICATION INTERFACES 11
4. ADDITIONAL NONFUNCTIONAL REQUIREMENTS 11
APPENDICES 13
APPENDIX A: GLOSSARY OF TERMS 13
APPENDIX B: ANALYSIS DOCUMENTATION 13
APPENDIX C: ISSUES 15

1. INTRODUCTION

1.1 PURPOSE

Hermes Platform application is an application for the Potchefstroom campus Courier office that will be used to help automate and speed up the process conducted by staff members. This will allow for a smoother process that will be able to grow with the increasing number of parcels that they are receiving. The system will be developed for laptops and desktops with a windows operating system.

1.2 INTENDED AUDIENCE AND PERTINENT SECTIONS

Developers' Crucial Areas:

Technical Specifications: These comprise thorough explanations of the technology stack, databases, APIs, and other technical elements of the application, as well as the architecture of the program.
Software Interfaces: Information about how to integrate the app with external systems and APIs
Operating Environment: Details about the hardware requirements and necessary operating systems.
Security Requirements: Guarantees that data stored in the database are protected and secured.

Testers' Crucial Areas:

External Interface Requirements: Testers are informed about expected behaviors, interface layouts, and user interactions in this part so that they can test the system and verify if the application is working in the expected way.
Performance Requirements: Information on anticipated KPIs for app performance, including handling of concurrent users, data transfer rates, and response times.
Software Quality: Assists in creating test cases that determine whether the program satisfies predetermined standards for maintainability and usability.

Users' Crucial Areas:

User Interfaces: Detailed explanations of how an application should seem and work from the viewpoint of the user.
Features: Being aware of the capabilities and advantages of the software.

1.3 PROJECT SCOPE

The Benefits of Hermes Platform:

- The application will allow for the automation of work which will lead to a decrease in time spent on admin tasks, such as generating reports, collecting parcels and delivery of parcels.
- The application will furthermore be able to generate reports for Potchefstroom campus Courier office such as total income, total parcels in a section of storage, etc.

Hermes Platform primary objectives are**:**

- Ease of use: create a system that will be easy to use by staff members of Potchefstroom campus Courier office that has logical sense and an easy-to-understand layout.
- Automation: the application will allow for the automation of certain tasks such as finding where the parcel is stored, and the sending of notifications the moment the parcel is ready for collection.

1.4 DOCUMENT CONVENTIONS

This document follows these conventions:

- Heading levels are indicated by heading styles (Heading 1, Heading 2, etc.)
- Bulleted lists are used to present sets of related items or options.
- Numbered lists are used to present sequential steps or instructions.
- When covering database aspects of the project an underscore will be used to better match the term will refer to in the actual database itself

1.5 REFERENCES
Feasibility study Hermes Platform.doc
Business case – Hermes Platform.doc
Functional specification – Hermes Platform.doc

2. DESCRIPTION

2.1 PRODUCT PERSPECTIVE
Potchefstroom campus Courier office currently receives and manages a growing volume of incoming courier packages using Microsoft SharePoint together with a largely manual administrative process.

Staff currently manually log every parcel and search for every parcel without the use of any electronic system. Total income can only be calculated through manual work of paging through all receipts that occur in a month and then adding all their totals together.

The current parcel volume already leads to lengthy admin task times and are inefficient and if the volume were to grow then the current system will go from tedious and time-consuming but functioning, all the way to no longer meeting the needs of the client or the client's clients.

2.2 FEATURES
The Hermes Platform system provides three core features that directly address the inefficiencies of the client's current manual, SharePoint-based process:

- **Parcel Management** — enables staff to add, search, filter, and remove parcel records, replacing manual logging and paper-based tracking.
- **Courier Management** — enables staff to maintain a list of the courier companies the client works with.
- **Automated Notification System** — automatically notifies a recipient once their parcel is ready for collection, removing the need for staff to contact each client manually.

2.3 USER OVERVIEW
The Client (Head Office)
Characteristics:
- Will use the system to perform admin work such as creating reports
- They value ease of use, organization, and accessibility.

Needs:
- Manage all staff members and their details as well as couriers
- Efficient report generation to make managerial decisions
- The ability to view and manage collection information reliably and timeously.

The client's (Potchefstroom campus courier) staff
Characteristics:
- Employed by the client
- Will use the system to perform admin work such as handing over parcels
- They value ease of use, organization, and accessibility.

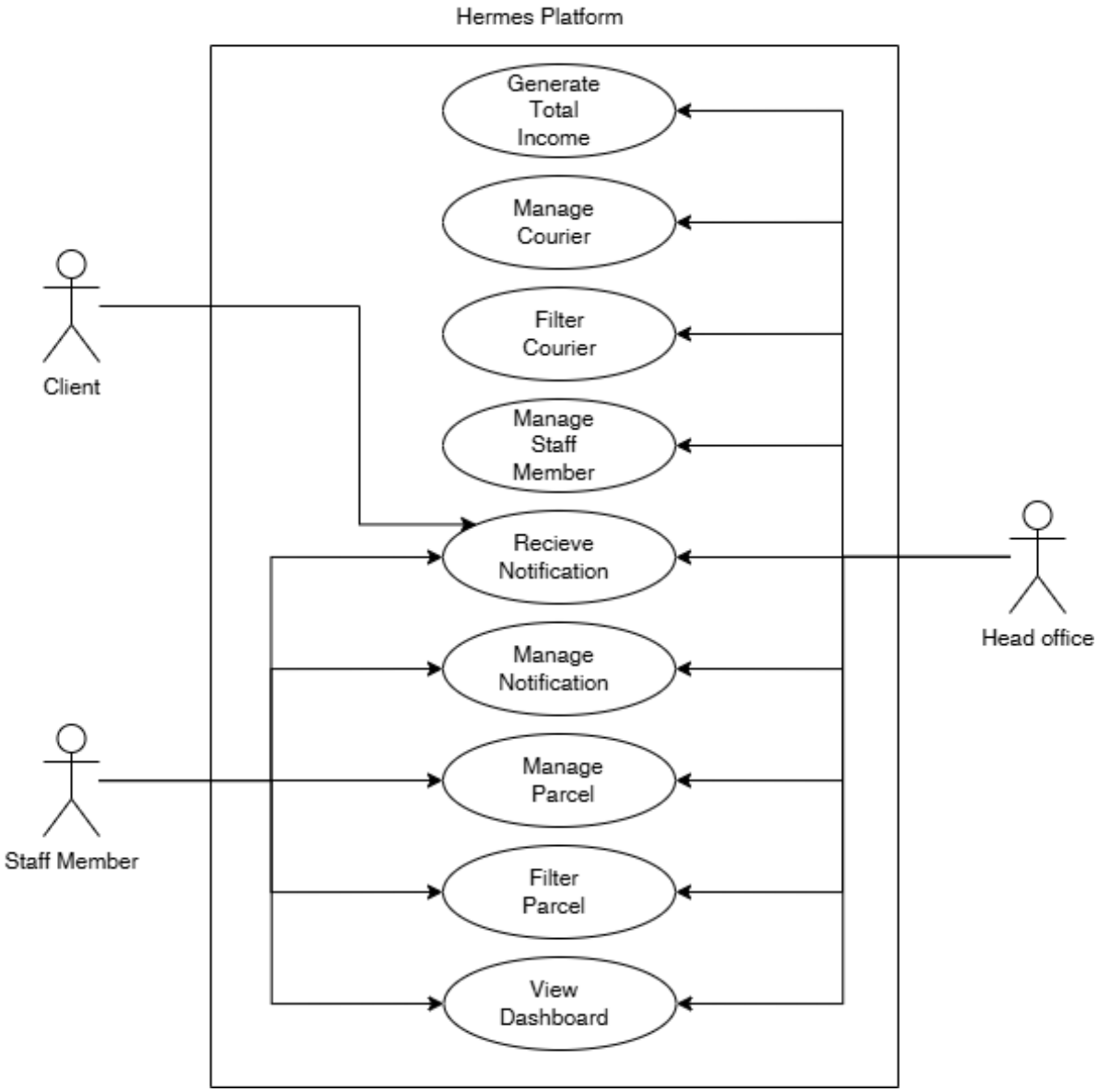
Needs:
- A user-friendly way to add and catalogue new parcels.
- Efficient search and filtering functionalities to locate specific parcels in their collection.
- The ability to view and manage collection information reliably and timeously.

The client's clients
Characteristics:
- These are people who have hired the client to transport parcels

- They value ease of use and expeditious delivery

Needs:

- To be informed when parcels can be collected
- Collect their parcels without any major delays



2.4 OPERATING ENVIRONMENT

Hardware Platforms

- The application is expected to run laptops and desktops from both the lower end to the higher end of specifications for these devices

Operating Systems

- The operating system used for the laptops and desktops will be

Other Software Components or Applications

- The system's core functionality does not require external systems or APIs beyond a notification delivery service used to inform clients when their parcels are ready for collection (e.g. an email delivery service or SMS/WhatsApp gateway — specific provider to be confirmed with the back-end team). No other external system integration is required for core operation.

2.5 CONTRAINTS: IMPLEMENTATION / DESIGN

Technical Difficulties

- The primary technical risk relates to the reliability and rate limits of the third-party notification provider (email/SMS/WhatsApp). Downtime or delays on the provider's side could delay notifications reaching clients. A manual fallback — staff contacting recipients directly — will remain available if the automated notification system experiences an outage.

Limitations on Resources

- Budget Restrictions: Developing high-quality software involves a significant financial commitment, especially in areas like server infrastructure, database security, and user interface design. Budgetary restrictions may restrict the range of features or the final product's quality.

Time Restrictions

- Development Time: It can take a long time to create sophisticated software with lots of features and a faultless user interface as such the system must be developed in a methodical manner to adhere to Northwest University's academic schedule.

2.6 DOCUMENTATION

Content

The application's content consists of all the data and media that will be made accessible to users, such as:

- User Interface Text: Any text in an application that directs the user on how to use various features, such as labels and instructions.
- FAQs & Support Documentation: Detailed instructions and answers to frequently asked questions to help users get the most out of the program, solve problems, and navigate it.
- Digital Content: This consists of the values found within the database tables found within the system, such as recipient_name, storage_location, courier_name, etc.
- No additional content is required beyond the system's UI text, FAQ/support documentation, and the database-driven digital content described above.

Mode of Delivery

- Installation file onto laptops of desktops using the Windows operating system.

Standards

- User Interface Standards: Ensuring the software is accessible and intuitive by adhering to best practices in UI/UX design.
- Technical Guidelines: The system will be built using PHP (Laravel framework) for the back end, providing a secure, well-supported server-side foundation, with TypeScript and React for the front end, rendered through Inertia.js for a smooth single-page-application experience without a separate REST API layer. The front-end UI uses Tailwind CSS with the shadcn/ui component library (built on Radix UI primitives) for consistent, accessible components. MySQL is the intended database. These are all actively maintained that industry-standard technologies chosen to support efficiency, security, and long-term stability.
- Quality Assurance: throughout the development phase there will be multiple testing opportunities where the system will be tested against expected behavior to monitor quality assurance and standards.

2.7 ASSUMPTIONS / DEPENDENCIES

Third-Party API Integration

Assumption: The system will depend on a third-party service to send notifications (email/SMS/WhatsApp) to clients' clients when parcels are ready for collection. The specific provider

is still to be confirmed with the back-end team.
Impact: If the chosen provider experiences downtime, rate limiting, or pricing changes, this could delay notification delivery or increase operating costs. The manual fallback noted above mitigates this risk.

Technological Progress

Assumption: After the application is released, current technology will continue to be reliable enough to support its development and operation without necessitating major adjustments or upgrades.
Impact: assumptions have been made that if proven incorrect will lead to either the lifetime of the product or its scalability to decline

SYSTEM FEATURES

Parcel Management

DESCRIPTION AND PRIORITY	Allows staff to create, view, search, filter, and delete parcel records, replacing the current manual logging process. Priority: High (core feature).
STIMULUS / RESPONSE SEQUENCES	Staff submit a new parcel record → system validates input, stores the record, and displays confirmation. Staff enters search/filter criteria → system returns matching records. Staff select a parcel to delete → system prompts for confirmation, then remove the record.
FUNCTIONAL REQUIREMENTS	REQ-1.1 System shall allow staff to add a parcel record (recipient name, contact details, storage location, status). REQ-1.2 System shall validate required fields before saving. REQ-1.3 System shall allow searching/filtering by recipient name, storage location, and status. REQ-1.4 System shall require confirmation before deleting a record.

Courier Management

DESCRIPTION AND PRIORITY	Allow staff to maintain the list of courier companies the client works with. Priority: Medium.
STIMULUS / RESPONSE SEQUENCES	Staff adds a new courier → system stores and displays it in the courier list. Staff select a courier to remove → system prompts for confirmation, then removes it.
FUNCTIONAL REQUIREMENTS	REQ-2.1 System shall allow staff to add a courier record. REQ-2.2 System shall display a list of all active couriers. REQ-2.3 System shall require confirmation before removing a courier.

Automated Notification System

DESCRIPTION AND PRIORITY	Automatically notifies a recipient once their parcel is marked ready for collection. Priority: High.
STIMULUS / RESPONSE SEQUENCES	Parcel status is updated to "ready for collection" → system automatically sends a notification and logs it in the database.
FUNCTIONAL REQUIREMENTS	REQ-3.1 System shall trigger a notification automatically when parcel status changes to "ready for collection." REQ-3.2 System shall log each notification sent, including timestamp and recipient. REQ-3.3 System shall support delivery via at least one channel (email/SMS/WhatsApp) provider pending confirmation with back-end team.

3. REQUIREMENTS OF EXTERNAL INTERFACE

3.1 USER INTERFACES

Style Guides and Standards

The user interface will be built using shadcn/ui, a component library built on Radix UI primitives, styled with Tailwind CSS v4. This provides a consistent, accessible, and modern set of components (buttons, dialogs, dropdowns, forms, etc.) across the application. The UI will be responsive and provide consistent experience across the range of laptop and desktop screen sizes the application targets.

Constraints

Device Compatibility: As the application targets laptops and desktops running Windows, the UI must render correctly across a range of screen resolutions and window sizes. Navigation uses a collapsible sidebar pattern that adapts to available screen width.

Functionality

Navigation: Users navigate via a persistent, collapsible sidebar with icon-only collapsed state, plus breadcrumb trails on each page, giving a consistent structure throughout the application.
Interactivity: Buttons, dropdown menus, dialogs, and toggles give immediate visual feedback (loading, disabled, hover/focus states) via Radix UI's built-in interaction patterns.
Forms: Validation errors display inline immediately after submission, using Inertia's form handling for real-time feedback without full page reloads.

Sample Screens

- Login Screen
- Registration Screen
- Forgot Password / Reset Password Screens
- Dashboard (Home) Screen
- Add Parcel Screen
- View Parcel Details Screen
- Browse/Search Parcels Screen
- Add Courier Screen
- View Courier Details Screen
- Notifications Screen
- User Profile / Account Settings Screen

3.2 HARDWARE INTERFACES

- No special or dedicated hardware is required to run Hermes Platform. The application runs on standard laptop and desktop hardware already available to Potchefstroom campus Courier office staff, using standard input devices (keyboard, mouse, trackpad) and a standard display.
- If a barcode scanner is adopted in future (per the budget consideration noted in Section 2.5), it would connect via a standard USB/HID interface and would not require dedicated drivers beyond what the Windows OS already supports.

3.3 SOFTWARE INTERFACES

Describe product / other software interface characteristics, including component names and versions, databases, operating systems, libraries, tools, etc. Specify any constraints, along with nature of communications and what data is coming in and being disseminated.

Hermes Platform is built on the following software components:

- **Back-end**: Laravel 12 (PHP 8.2+), handling routing, business logic, and authentication.

- **Front-end**: React 19 with TypeScript, rendered through Inertia.js v2, which passes data directly from Laravel controllers to React page components — removing the need for a separate REST API layer between front end and back end.
- **UI Components**: shadcn/ui, built on Radix UI primitives, styled with Tailwind CSS v4, for consistent and accessible interface elements.
- **Build Tooling**: Vite 7, for bundling and serving front-end assets in both development and production.
- **Database**: MySQL, storing parcel, courier, user, and notification records.
- **Routing Bridge**: Ziggy, which exposes Laravel's named routes to the TypeScript front end.
- **Constraint**: Because Inertia.js binds the front end directly to Laravel's routing and session state, the front end is not independently deployable — it is not a fully decoupled single-page application/API architecture, and both layers are developed and released together.
- **Data flow**: Requests between front end and back end are standard HTTP requests carrying JSON payloads. Laravel controllers query MySQL via the Eloquent ORM and return data as page props, which React renders directly — there is no polling or separate API gateway involved.

3.4 COMMUNICATION INTERFACES

List requirements of communication functions: email, browsers, servers, forms, etc. Describe standards employed, security or encryption measures, data transfer rates, and synching.

- **Notification Delivery**: The system sends an automated notification to a client's client when their parcel is marked ready for collection, via [email / SMS / WhatsApp — specific channel(s) to be confirmed with the back-end team].
- **Standards**: Notifications are sent through the provider's standard integration (e.g. SMTP for email, or a messaging API for SMS/WhatsApp), authenticated via API key as required by the provider.
- **Security**: Notification provider credentials are stored as environment variables, never committed to source control, and all communication with the provider uses HTTPS/TLS encryption.
- **Data Transfer**: Notifications are triggered synchronously the moment a parcel's status changes to "ready for collection." Each notification's delivery status is logged in the database for staff visibility — no continuous polling or synching is required, as communication is event-driven.
- **Internet Dependency**: Sending a notification requires an active internet connection; core parcel/courier record-keeping is not affected by connectivity to the notification provider.

4. ADDITIONAL NONFUNCTIONAL REQUIREMENTS

NFR 1 Responsiveness:
The application should be responsive and provide smooth user interaction. This includes minimal loading times when searching for the collection, adding new parcels, or using other functionalities.

NFR 2 Efficiency:
The application should operate smoothly on a range of laptops and desktops from both the lower end all the way to a higher end.

NFR 3 Data Security:
User data, such as collection details and login credentials, should be stored securely on the device and not break POPIA.

NFR 4 User Authentication:
Authentication methods must be put in place to prevent unauthorized access to the application.

NFR 5 Usability:
Within the first month of deployment the client should indicate satisfaction with product delivered.

NFR 6: Maintainability
90% of defects or issues should be fixed in a week once the product has been deployed

NFR 7 Verification
To monitor response times and efficacy, keep a record of issue reports and fixes.

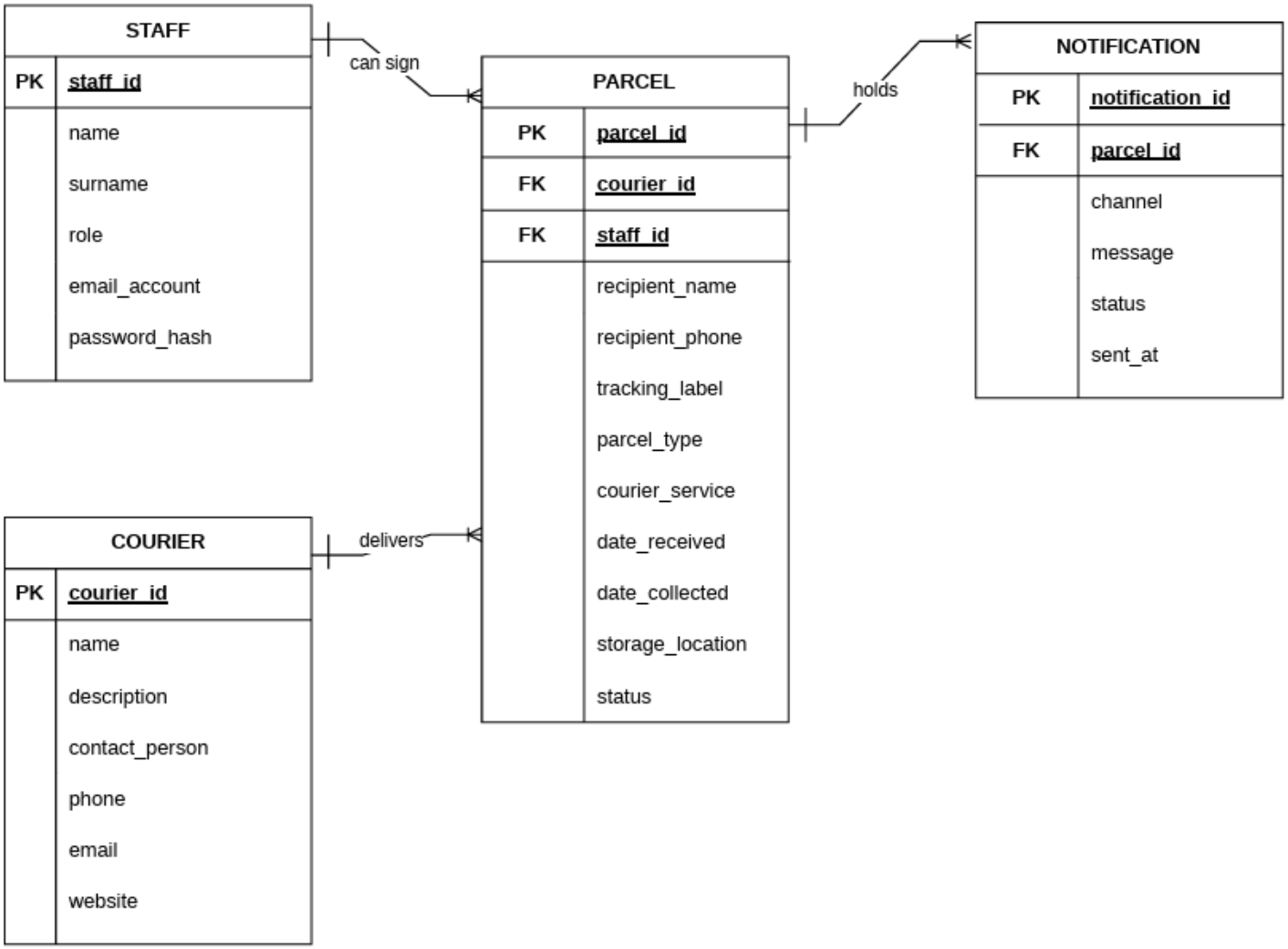
APPENDICES

APPENDIX A: GLOSSARY OF TERMS

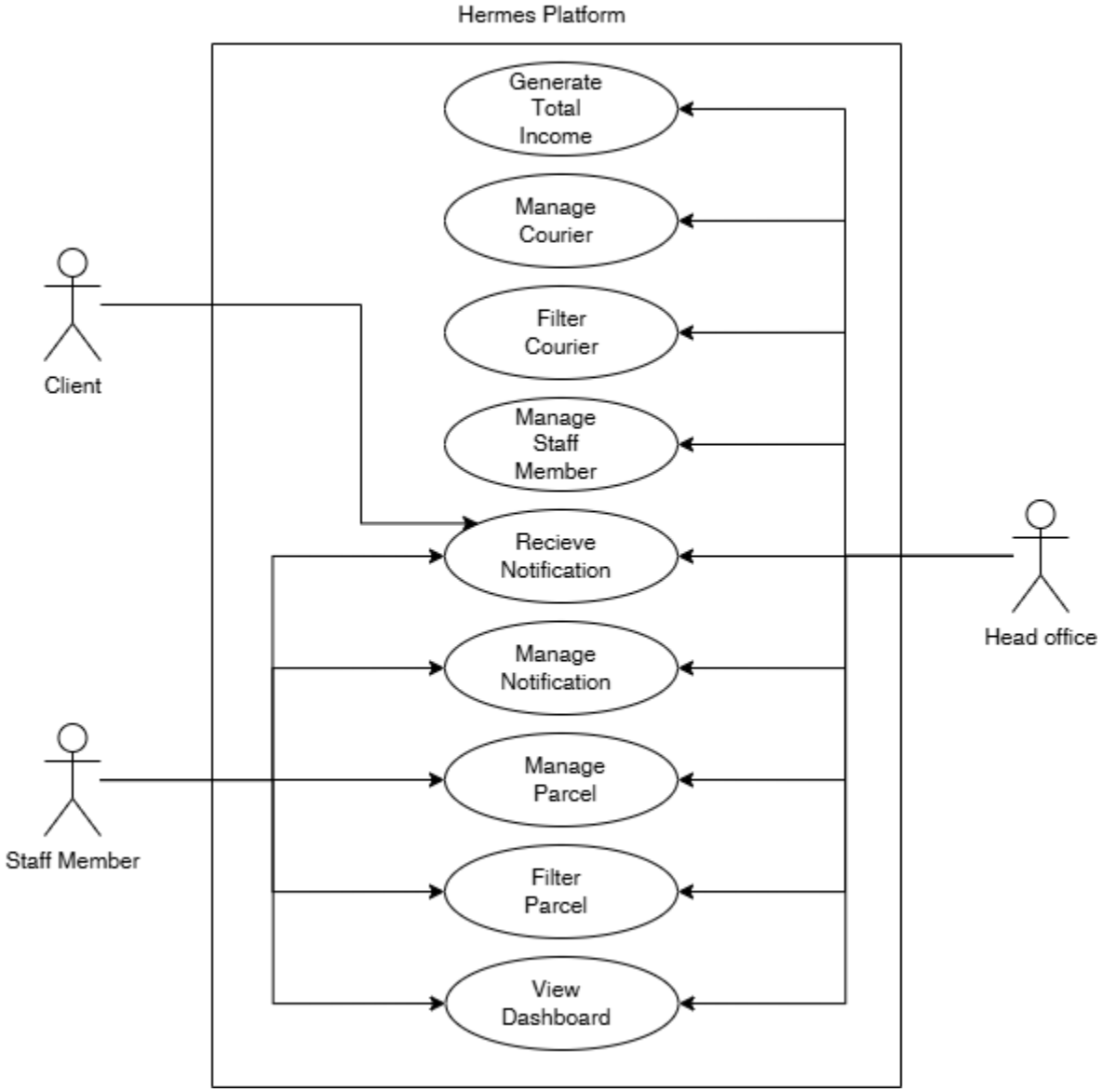
Client refers to the Potchefstroom campus courier.
Team refers to the entire development team including the project manager, frontend development, backend development and documentation teams.
Application refers to the Hermes platform system.
NWU refers to Northwest University.
OS refers to Operating system

APPENDIX B: ANALYSIS DOCUMENTATION

HERMES_ERD.pdf



Hermes_Use_Case.pdf



APPENDIX C: ISSUES

ISSUES		
ID	**DESCRIPTION**	**PARTY RESPONSIBLE**
ISS-001	Final confirmation of barcode scanner hardware model and compatibility.	Open
ISS-002	Confirmation of exact flat fee amount for automated income reporting.	Open
ISS-003	Final selection of South African bulk SMS provider and API integration details.	Open